API Contract — Adjudicator Lab

Tài liệu hợp đồng giữa front-end và back-end. Mọi component phía client đều gọi API qua `src/lib/api` (`api.*`); không component nào gọi `fetch` trực tiếp. Kiểu dữ liệu (DTO) nằm ở `src/lib/api/types.ts`. Khi backend đổi hợp đồng, sửa file đó trước, rồi tới tài liệu này.

Hiện các Route Handler trong `src/app/api/**` là **mock** (dữ liệu fixture, lưu trong bộ nhớ hoặc JSONL). Backend cần cài đặt thật đúng các hợp đồng dưới đây.

Quy ước chung

Mục	Quy ước
Base path	`/api` (cùng origin, qua Next.js Route Handlers — có thể proxy sang service backend)
Định dạng	JSON (`Content-Type: application/json`), trừ các ngoại lệ ghi rõ: NDJSON, SSE, CSV, upload JSONL
Lỗi	Status ≠ 2xx với body `{ "error": string, "code"?: string }`. `error` là thông điệp tiếng Việt hiển thị thẳng cho người dùng. `ApiError` ở client đọc `error` và `code`
`grading`	**Bắt buộc** loại trường `grading` khỏi mọi response (`stripGrading()` trong `src/guard/grading-proxy.ts`)
API keys	Không bao giờ nhận hay trả API key của nhà cung cấp mô hình. Key chỉ đặt ở biến môi trường máy chủ
Huỷ request	Client gửi `AbortSignal`; backend nên chịu được việc client ngắt kết nối giữa chừng

Trạng thái trong các bảng: - **Có sẵn**: route đã tồn tại từ trước, front-end chỉ đổi sang gọi qua `api.*`. - **Mới**: route mới thêm trong đợt refactor này, hiện dùng mock (`src/mock/*`). Backend cần cài đặt thật. - **Đổi**: route có sẵn nhưng hành vi đã thay đổi.

Tổng quan endpoint

#	Method	Path	Trạng thái	Front-end dùng ở
1	GET	`/api/datasets`	Có sẵn	(chưa dùng ở UI)
2	POST	`/api/datasets`	Có sẵn	(chưa dùng ở UI)
3	GET	`/api/datasets/{datasetId}/tasks`	Có sẵn	`LibraryScreen`, `ChatTestbed`
4	GET	`/api/problems`	Có sẵn	(chưa dùng ở UI)
5	GET	`/api/runs`	Có sẵn	`HistoryScreen`
6	POST	`/api/runs`	Có sẵn	`ChatTestbed` (mode evaluation)
7	DELETE	`/api/runs`	Có sẵn	`HistoryScreen`
8	GET	`/api/runs/{runId}`	Có sẵn	`RunDetailScreen`
9	PATCH	`/api/runs/{runId}`	Có sẵn	`ChatTestbed`
10	PATCH	`/api/runs/{runId}/review`	Có sẵn	`RunDetailScreen` → `ReviewForm`
11	POST	`/api/runs/{runId}/followup`	Có sẵn	`ChatTestbed`
12	GET	`/api/runs/export?format=`	Có sẵn	`HistoryActions` (link tải)
13	GET	`/api/logs/stream?runId=` (SSE)	Có sẵn	`LiveLogPanel`
14	GET	`/api/stats`	Có sẵn	`StatsScreen`
15	GET	`/api/runs/{runId}/manifest`	Có sẵn	`LibraryScreen`
16	POST	`/api/runs/{runId}/held-out`	Có sẵn (body mới)	`LibraryScreen` → `HeldOutPartitionCard`
17	GET	`/api/runs/{runId}/events` (SSE)	Có sẵn	`AdmissionScreen` (`useRunEvents`)
18	GET	`/api/cases`	**Mới**	`ChatTestbed`, `RepairGatePanel`
19	GET	`/api/cases/{caseId}`	Có sẵn	(dự phòng, `api.cases.get`)
20	POST	`/api/cases/{caseId}/repair/{repairId}`	Có sẵn	`RepairGatePanel`
21	POST	`/api/runs/{runId}/cases/{caseId}/stream` (NDJSON)	Có sẵn	`useCaseStream` → `ChatTestbed`
22	POST	`/api/tickets/{ticketId}/answer`	**Đổi**	`AnnotateScreen`, `ChatTestbed`
23	POST	`/api/tickets/{ticketId}/defer`	**Đổi**	`AnnotateScreen`
24	GET	`/api/runs/{runId}/annotation-queue`	**Mới**	`AnnotateScreen`
25	GET	`/api/runs/{runId}/admission`	**Mới**	`AdmissionScreen`

#	Method	Path	Trạng thái	Front-end dùng ở
26	POST	`/api/runs/{runId}/admission`	**Mới**	`AdmissionScreen` (nút chạy kiểm nạp)
27	POST	`/api/runs/{runId}/admission/advise`	**Mới**	`AdmissionStepCard`
28	GET	`/api/runs/{runId}/evidence`	**Mới**	`EvidenceScreen`
29	GET	`/api/runs/{runId}/revocation`	Có sẵn	`RevocationScreen`
30	POST	`/api/runs/{runId}/revocation`	Có sẵn	`RevocationScreen`
31	GET	`/api/runs/{runId}/results/{id}`	**Mới**	`ResultsScreen` → `RQChart`, `ChartContainer`
32	GET	`/api/experiments/options`	**Mới**	`ExperimentsScreen` → `ExperimentConfigForm`
33	POST	`/api/experiments/plan`	**Mới**	`ExperimentsScreen` → `CostEstimatePanel`
34	POST	`/api/experiments`	**Mới**	`useBatchRun`
35	GET	`/api/experiments/{experimentId}`	**Mới**	`useBatchRun` (poll 500 ms)
36	GET	`/api/fault-injection`	**Mới**	`FaultInjectionScreen`
37	POST	`/api/fault-injection`	**Mới**	`FaultInjectionScreen`
38	GET	`/api/settings`	**Mới**	`SettingsScreen`
39	PUT	`/api/settings`	**Mới**	`SettingsScreen`
—	GET	`/api/mock`	Placeholder	Không dùng, nên xoá khi có backend thật

---

1. Datasets & Tasks

1.1 `GET /api/datasets`

Danh sách dataset, mỗi dataset kèm toàn bộ task.

200 `DatasetListResponse`


```
{ "datasets": [{ "id": "evalplus_illustrative", "name": "EvalPlus Tập Minh Họa (12 bài)", "t
```


1.2 `POST /api/datasets` — upload JSONL

- Body: text thô, mỗi dòng một `AcademicTask` (`Content-Type: application/x-ndjson`).
- Validate bằng `AcademicTaskSchema` (`src/core/schema.ts`).
- **201** `{ success: true, message, taskCount, tasks }`

- **400** body rỗng
- **422** `{ success: false, totalLines, validCount, errorCount, errors: [{ line, field?, message }] }`

1.3 `GET /api/datasets/{datasetId}/tasks`

200 `DatasetTasksResponse` → `{ datasetId, taskCount, tasks: AcademicTask[] }` · **404** dataset không tồn tại.

`AcademicTask`:

```
{ id: string; name: string; intentDocstring: string;
  outputType: "int" | "str" | "list" | "float" | "bool" | "other";
  isUniqueOutput: boolean; assignedBy: string; anchorOrigin: "gốc" | "dẫn xuất";
  epsilon?: number; hasA2: boolean; hasA3: boolean;
  anchorLevel: "A0" | "A1" | "A2" | "A3"; isBSynthSupported: boolean }
```

1.4 `GET /api/problems`

`{ datasetId, problems: Problem[] }`, với `Problem = { id, title, prompt, group?, difficulty?, goldLabel? }`. UI chưa dùng.

---

2. Evaluation runs (Kiểm thử · Lịch sử · Thống kê)

Hiện lưu ở `.data/evaluation-runs.jsonl` (`src/core/evaluation-store.ts`). Backend nên chuyển sang DB.

`Run` (`src/core/evaluation-types.ts`):

```
{ run_id: string; created_at: ISO; updated_at: ISO;
  dataset: { id: string; version: string }; problem_id: string; problem: string;
  spec?: AgentOutput; code?: AgentOutput; turns: Turn[]; verdicts: EvalVerdict[];
  confidence?: number; latency: Record<string, number>; gold_label?: EvalVerdict;
  logs?: RunLog[]; status: "queued" | "spec_running" | "code_running" | "judging" | "answere
  review?: { agree: boolean; note: string; at: ISO } }
```

2.1 `GET /api/runs`

Query (đều tuỳ chọn): `page` (mặc định 1), `pageSize` (1–100, mặc định 25), `verdict` (khớp chuỗi con, không phân biệt hoa thường, so với verdict cuối), `problem` (khớp `problem_id + problem`), `dataset`, `from`, `to` (ISO date, so với `created_at`).

200 `RunListResponse` → `{ total, page, pageSize, runs: Run[] }`, sắp `created_at` giảm dần.

2.2 POST /api/runs

Endpoint này có **hai chế độ** (backend nên tách thành hai endpoint riêng):

a) Evaluation run (ChatTestbed dùng):

```
{ "mode": "evaluation", "datasetId": "evalplus_illustrative", "datasetVersion": "12 bài · mo
```

→ **201** { run: Run } (status queued, có sẵn 1 turn user).

b) Khởi tạo run thực nghiệm có chia held-out: { datasetId?, seed?, heldOutRatio? } → **201** RunManifest · **400** nếu heldOutRatio ∉ (0,1).

2.3 DELETE /api/runs

Body { "confirmation": "XOA LICH SU" } → **200** { deleted: true } · **400** sai cụm xác nhận.

2.4 GET /api/runs/{runId}

200 { run: Run } · **404**.

2.5 PATCH /api/runs/{runId}

Ghi kết quả phân xử vào run. Mọi trường đều tuỳ chọn:

```
{ patch?: Partial<{ status; confidence; gold_label }>;
  turn?: { role: "user" | "system"; content: string; verdict?: EvalVerdict; confidence?: num
  verdict?: EvalVerdict;                  // append vào verdicts[]
  latency?: Record<string, number>;       // merge vào latency
  log?: string; level?: "info" | "warn" | "error" }  // append vào logs[]
```

→ **200** { run: Run } · **404**.

□ Mock hiện spread body.patch thẳng vào run. Backend **phải whitelist** các trường được phép sửa.

2.6 PATCH /api/runs/{runId}/review

Body { agree: boolean, note: string } → **200** { run } · **400** sai kiểu · **404**.

2.7 POST /api/runs/{runId}/followup

Lượt bổ sung **duy nhất** khi verdict UNKNOWN. Body { content: string } (không rỗng). **200** { run } (status followup) · **400** rỗng · **404** · **409** đã dùng lượt bổ sung.

2.8 GET /api/runs/export?format=csv|json|stats

Tải file (Content-Disposition: attachment). Mặc định json. CSV gồm các cột: run_id, created_at, dataset, problem_id, problem, verdict, confidence, status, gold_label, review_agree, review_note.

2.9 GET /api/logs/stream?runId= — SSE

Mỗi message: data: RunLog\n\n với RunLog = { id, at, level, message }. Server đóng stream khi run ở trạng thái final hoặc error. **400** thiếu runId.

2.10 GET /api/stats

Query: dataset, verdict, from, to. **200** StatsResponse → { metrics: Metrics, runs: Run[] }

```
Metrics = { totalRuns; byVerdict: Record<string, number>; accuracy?; averageConfidence?;
            latency: { mean?; p50?; p95? }; followupsUsed; unknownResolvedRate? }
```

Các biểu đồ còn lại (theo đề, theo ngày, ma trận nhầm lẫn, histogram confidence) hiện tính ở client (src/features/stats/derive.ts). Nếu số run lớn, backend nên trả sẵn các tổng hợp này để khỏi gửi toàn bộ runs.

---

3. Held-out partition

3.1 GET /api/runs/{runId}/manifest

200 RunManifest · **404**.

```
RunManifest = { id; datasetId; seed; heldOutRatio; totalTasks; runTaskIds: string[]; heldOut
                isHeldOutLocked: boolean; activeMonitorVersion: string; createdAt: ISO }
```

LibraryScreen đọc isHeldOutLocked để hiện trạng thái khoá ngay khi mở trang.

3.2 POST /api/runs/{runId}/lock-heldout

Body (**mới**, front-end đã gửi): { seed?: string, heldOutRatio?: number } — seed và tỷ lệ người dùng chọn trên UI. **200** { success: true, message, manifest: RunManifest } · **400** { error } (đã khoá, run không tồn tại…).

> TODO backend: mock hiện **bỏ qua** seed/heldOutRatio trong body. Nếu run chưa phân chia thì dùng hai giá trị này để chia tất định (partitionTasksDeterministically) trước khi khoá. Sau khi khoá thì không được ghi đè nữa.

3.3 GET /api/runs/{runId}/events — SSE có thể nối lại

- Mỗi event: id: <seq>\nevent: <type>\ndata: <json>\n\n, seq tăng dần từ 1.

- Nối lại bằng header `Last-Event-ID` hoặc query `?lastEventId=`: chỉ phát các event có `seq > lastEventId`, không mất, không trùng.
- `type  run_created | admission_step | admission_completed | verdict | heldout_locked | repair_deployed | patch_revoked`.
- **404** run không tồn tại.

---

4. Cases, Repair & luồng phân xử

`CaseDossier` (`src/core/types.ts`): `{ id, taskId, witness, codeOutput, specPO, monitorRejectionClause?, isFlaky?, groundedEvidence[], advisoryEvidence[], adjudication: { verdict, matchedRule, explanation, ... }, ticket?: HumanQueryTicket, repairCandidate?, gateResult?, currentMonitorVersion }`.

4.1 `GET /api/cases` — Mới

Query: `hasTicket=true` để chỉ lấy case có vé hỏi người. **200** `CaseListResponse` → `{ cases: CaseDossier[] }`. Dùng cho danh sách case trong `ChatTestbed` (xoay vòng khi bấm "Phiên mới") và ô chọn case trong `RepairGateScreen`.

> TODO backend: nên nhận thêm `runId` để lọc case theo run, và phân trang khi số case lớn.

4.2 `GET /api/cases/{caseId}`

200 `CaseDossier` · **404**.

4.3 `POST /api/cases/{caseId}/repair/deploy`

Body `DeployRepairRequest`:

```
{ repairCandidate?: RepairCandidate;   // bỏ trống thì dùng candidate có sẵn trong dossier
  policy: "strict" | "standard" | "shadow";
  isProofVerified?: boolean; hasA2Anchor?: boolean }
```

200 `DeployRepairResponse`:

```
{ success; caseId; policy; passMode?; policyDecision: { canDeploy: boolean; reason?: string;
  isShadowOnly; gateResult: GateResult; previousMonitorVersion; activeMonitorVersion }
```

400 không có candidate · **404** case không tồn tại. Front-end coi `success: false` là bị chính sách từ chối và hiển thị `policyDecision.reason`.

4.4 `POST /api/runs/{runId}/cases/{caseId}/stream` — NDJSON

Body `{ prompt?: string }`. Response `Content-Type: application/x-ndjson`; mỗi dòng là một `DualOracleStreamEvent` (`src/core/stream-types.ts`):

```
| { type: "step"; id: StreamStepId; status: "running" | "done" | "error"; label: string; dur
| { type: "text"; delta: string }                          // token giải thích
| { type: "human_query"; reason: "UND_E" | "UND_C" | "UND_G"; promptText: string; questionTy
| { type: "done"; verdict: Verdict; totalMs: number }
| { type: "error"; message: string; step?: StreamStepId }
```

Thứ tự bước: `load_task → run_code → oracle_p0 → oracle_code → adjudicate → fail_closed_check`.

> **Đổi phía client:** trước đây khi stream lỗi, client tự **giả lập** toàn bộ kết quả từ fixture. Phần giả lập đã bị bỏ: lỗi mạng/API giờ hiện thành lỗi kèm nút "Thử lại". Backend phải bảo đảm endpoint luôn kết thúc bằng `done` hoặc `error`.
>
> TODO backend: mock hiện bỏ qua `prompt` và chỉ phát lại dữ liệu của case. Bản thật cần chạy oracle trên prompt người dùng.

---

5. Human query tickets & Annotation (Focus Mode)

5.1 `POST /api/tickets/{ticketId}/answer` — Đổi

Body `TicketAnswerRequest`:

```
{ answer?: unknown;          // true = Chấp nhận, false = Bác bỏ, hoặc chuỗi người dùng nhập
  isUncertain?: boolean;     // MỚI: "Không xác định được" - vẫn tiêu vé vĩnh viễn
  durationMs?: number }      // thời gian trả lời (đo năng suất gán nhãn)
```

- **200** `{ success: true, ticket: HumanQueryTicket (status "spent"), message }`
- **404** vé không tồn tại
- **409** `{ error, code: "TICKET_ALREADY_SPENT" }` — **single-shot**: mỗi vé chỉ được trả lời một lần

Thay đổi: hỗ trợ `isUncertain` (lưu answer = `{ uncertain: true }`). Trạng thái vé giờ nằm ở store dùng chung `src/mock/ticket-store.ts` để hàng đợi gán nhãn đọc cùng nguồn.

> TODO backend: `ChatTestbed` hiện **tự suy ra** verdict cuối sau lượt bổ sung (`UNKNOWN_EVIDENCE → ACC`, còn lại `SPEC_FAULT`, xem `followupVerdict()` trong `src/features/chat/ChatTestbed.tsx`). Đề xuất response trả thêm `resolvedVerdict` và `explanation` để client bỏ logic này.

5.2 `POST /api/tickets/{ticketId}/defer` — Đổi

Hoãn vé (giữ open, client đẩy về cuối hàng đợi). **200** `{ success: true, ticketId, status: "open", message }` · **404** (mới) vé không tồn tại.

5.3 `GET /api/runs/{runId}/annotation-queue` — Mới

Hàng đợi gán nhãn cho Focus Mode. **200** `AnnotationQueueResponse`:

```
{ runId: string; seed: string; total: number; items: CaseDossier[] }  // mọi item đều có ti
```

- Chỉ gồm các case có vé **chưa spent**; `ticket.status` lấy theo trạng thái hiện tại trong store.
- Thứ tự xáo trộn **tất định** theo `seed` (mock: `"annotation_seed_2026"`) để mọi người gán nhãn thấy cùng một thứ tự.
- Client giữ hàng đợi cục bộ sau lần tải đầu: trả lời thì bỏ phần tử đầu, hoãn thì đẩy về cuối.

---

6. Admission (Kiểm nạp 6 bước) — Mới

6.1 `GET /api/runs/{runId}/admission`

Báo cáo kiểm nạp gần nhất.

6.2 `POST /api/runs/{runId}/admission`

Chạy lại kiểm nạp và trả về báo cáo mới (body rỗng). Nếu pipeline chạy lâu, đề xuất trả **202** ngay rồi đẩy tiến độ qua SSE `events` (`admission_step`, `admission_completed`); khi đó front-end gọi lại `GET`.

200 (cả 6.1 và 6.2) `AdmissionReport`:

```
{ runId: string; ranAt: ISO;
  steps: Array<{ index: number; key: string; title: string; description: string;
                 status: "idle" | "running" | "passed" | "warning" | "error";
                 details?: string;
                 hasViolation?: boolean; violatedTaskId?: string;      // bước anchor cross
                 silentFaultsCount?: number; silentFaultCases?: string[]; // bước replay
                 slackRatio?: number }>;                               // bước B_synth (%)
  summary: { slackRatio: number; slackCases: number; totalTasks: number;
             silentFaultCases: string[]; admittedTasks: number; excludedTasks: number };
  reviewedTaskIds: string[] }   // các task vi phạm đã được đánh dấu để người kiểm lại
```

6 bước theo thứ tự `key`: `static_checker`, `contract_runner`, `anchor_cross_check`, `silent_fault_replay`, `z3_property_verifier`, `bsynth_slack_measurement`.

6.3 `POST /api/runs/{runId}/admission/review`

Body `{ taskId: string }` → **200** `{ runId, reviewedTaskIds: string[] }` · **400** thiếu `taskId`. Thao tác idempotent.

---

7. Evidence Store — Mới

7.1 GET /api/runs/{runId}/evidence

Kho bằng chứng **chỉ đọc**. UI không có thao tác ghi nào trên G/B; có test bảo vệ bất biến này.

200 EvidenceStoreResponse:

```
{ runId: string;
  good: EvidenceRecord[];      // Tập G
  bad: EvidenceRecord[];       // Tập B
  bSynth: BSynthRecord[];      // Tập B_synth
  metamorphicTree: MetamorphicAnchorNode;   // cây anchor gốc → dẫn xuất qua MR
  specDebt: SpecDebtItem[];
  invariant: { holds: boolean; intersectionCount: number; violatingKeys: string[] } }  // G
```

Kiểu chi tiết ở src/mock/fixtures/evidence-store.ts: - EvidenceRecord = { id, taskId, kind: "expected_output" | "property" | "rejection", source, input, output, verdict: "ACC" | "REJ", timestamp, notes? } - BSynthRecord = { id, taskId, mutationType: "boundary_shift" | "type_perturbation" | "extreme_value" | "null_injection", input, mutantOutput, isCaughtByPO, timestamp } - MetamorphicAnchorNode = { id, taskId, input, expectedOutput, origin: "gốc" | "dẫn xuất", parentId?, metamorphicRelation?, children[] } - SpecDebtItem = { id, taskId, witness, debtType: "unresolved_advisory" | "missing_a2_anchor" | "weak_precondition" | "open_ticket", title, description, severity: "high" | "medium" | "low", recordedAt }

Bất biến G ∩ B = ∅ **phải tính ở server** (khoá so sánh taskId:input->output). Client chỉ hiển thị kết quả.

---

8. Revocation (Thu hồi bản vá)

8.1 GET /api/runs/{runId}/revocation

200 { runId, monitorHistory: MonitorVersionRecord[] }

```
MonitorVersionRecord = { version: "P_0" | "P_1" | ...; patchId?; guard?; replacement?;
                         status: "active" | "revoked" | "deprecated"; deployedAt: ISO;
                         revokedAt?; revocationReason?; violatingTraceId? }
```

8.2 POST /api/runs/{runId}/revocation — tiêm vết xấu vào B

Body { trace: BadTraceItem } với BadTraceItem = { id, input, output, inGuard: boolean, description?, injectedAt: ISO }. **200** { success: true, revocationResult: { revoked: boolean; violatingTrace?;

`reason?; newActiveVersion? }, currentHistory: MonitorVersionRecord[] }`.

> TODO backend: mock trả 200 cho run không tồn tại. Nên trả 404 và validate `trace`.

---

9. Results RQ1–RQ6 — Mới

9.1 `GET /api/runs/{runId}/results/{rq}`

`rq` `rq1` … `rq6` (không phân biệt hoa thường). **404** nếu `rq` không hợp lệ.

200 `RQResultResponse`:

```
{ runId: string; rq: RQId; title: string; description: string; sampleSize: number;
  table: { headers: string[]; rows: Array<Array<string | number>> };    // chế độ "Xem dạng b
  data: RQData }                                                         // dữ liệu biểu đồ
```

`RQData` là union phân biệt theo `rq` (kiểu chi tiết ở `src/reporting/results-data.ts`):

rq	Payload `data`
rq1	`{ rq: "rq1", categories: RQ1Category[] }` — `{ category, beforeCount, afterCount, reductionRatio, witnesses[] }`
rq2	`{ rq: "rq2", methods: RQ2MethodData[], confusionMatrix: ConfusionMatrixRow[] }`
rq3	`{ rq: "rq3", cells: AblationCell[] }` — anchorLevel × source
rq4	`{ rq: "rq4", cells: ModelCorrelationCell[] }` — coder × spec
rq5	`{ rq: "rq5", suite: RQ5Metric[], unknownGateBreakdown: UnknownGateDecomposition[] }`
rq6	`{ rq: "rq6", strategies: RQ6Strategy[] }` — mỗi strategy có `points[]` (`residualRisk` theo số câu hỏi)

Mỗi điểm dữ liệu có thể mang `witnesses: LinkedWitness[]` (`{ caseId, taskId, witnessPreview, verdict, note? }`). Khi bấm vào một phần tử biểu

đồ, UI mở drawer witness và link `/runs/{runId}/workbench?caseId=…`. Backend phải giữ đúng `caseId` thật.

> TODO backend: hiện là hằng số minh hoạ (`src/reporting/rq-results.ts`), `sampleSize` cố định 164, `runId` bị bỏ qua. Cần tổng hợp từ kết quả thực nghiệm của run. Nhãn EvalPlus là nhãn vận hành; disclaimer này hiển thị cố định ở UI.

---

10. Experiments (Batch Runner) — Mới

`ExperimentConfig`:

```
{ datasetId: string; heldOutRatio: number /* (0,1) */; coderModel: string; specModel: string
  anchorLevel: "A0" | "A1" | "A2" | "A3"; policy: "strict" | "standard" | "shadow";
  unknownMode: "closed" | "open"; humanOracle: "off" | "perfect" | "noise_5" | "noise_10" |
  baselines: string[]; seed: string }
```

Validate bằng `ExperimentConfigSchema` (`src/mock/experiments.ts`).

10.1 `GET /api/experiments/options`

200 `ExperimentOptionsResponse`:

```
{ datasets: Array<{ id; name; taskCount; heldOutRatio; isHeldOutLocked }>;
  models: Array<{ id; name; family }>;       // family dùng để cảnh báo thiên kiến cùng họ
  baselines: string[];
  anchorLevels: ("A0" | "A1" | "A2" | "A3")[] }
```

10.2 `POST /api/experiments/plan`

Body `{ config: ExperimentConfig }`. Client gọi lại mỗi khi form đổi (request cũ bị huỷ). **200** `ExperimentPlanResponse`:

```
{ familyCheck: { isSameFamily: boolean; warning?: string };
  costEstimate: { functionCalls; llmDurationSec; z3DurationSec; totalTokens; estimatedUsd };
  manifest: { manifestHash; config; totalTasks; runTaskCount; heldOutTaskCount; initialMonit
```

400 `{ error }` cấu hình sai. Yêu cầu: **tất định**, cùng config thì cùng `manifestHash`.

10.3 `POST /api/experiments`

Body `{ config }` → **201** `ExperimentStatusResponse` · **400**.

10.4 `GET /api/experiments/{experimentId}`

Client poll mỗi 500 ms cho tới khi `status === "completed"`. **200** `ExperimentStatusResponse`:

```
{ experimentId: string; status: "running" | "completed"; config: ExperimentConfig; manifestH
  progress: { completed; total; accCount; rejCount; unknownCount; repairCount; revocations;
```

404 không tồn tại.

> TODO backend: chạy batch bằng job queue/worker và lưu tiến độ bền vững (mock giữ job trong bộ nhớ và tính tiến độ theo thời gian). Có thể thay polling bằng SSE. `pendingHumanTickets > 0` khi `humanOracle = "real_human"`: các vé này phải xuất hiện trong `/annotation-queue`.

---

11. Fault Injection — Mới

11.1 `GET /api/fault-injection`

200 `FaultInjectionStateResponse`:

```
{ options: { specFaults: Option[]; codeFaults: Option[]; trustedFaults: Option[] };  // Opt
  defaults: { specFault; codeFault; trustedFault };
  metrics: FaultInjectionMetrics }   // số liệu nền trước khi chạy
```

Giá trị hợp lệ hiện tại: - `specFault`: `over_tight` | `under_tight` | `semantic_flip` - `codeFault`: `off_by_one` | `operator_flip` | `extreme_input` - `trustedFault`: `none` | `mr_corrupt` | `docstring_corrupt` | `a3_conflict`

11.2 `POST /api/fault-injection`

Body `{ specFault, codeFault, trustedFault }` → **200** `{ profile, metrics: FaultInjectionMetrics, ranAt }` · **400** profile sai.

```
FaultInjectionMetrics = { misattributionRate: number /* % */; faultyPatchSlipRate: number /
                          gateBlockedCount: number; totalInjections: number }
```

> TODO backend: mock chỉ xét `trustedFault !== "none"`. Bản thật cần sinh đột biến theo cả 3 khu, chạy adjudicator và gate trên N ca rồi đo. Nếu chạy lâu, nên chuyển sang job + polling giống Experiments.

---

12. Settings — Mới

12.1 `GET /api/settings`

200 `SettingsResponse` → `{ settings: LabSettings, models: ModelOption[], updatedAt?: ISO }`.

12.2 `PUT /api/settings`

Body: `LabSettings` đầy đủ (ghi đè toàn bộ):

```
{ defaultCoderModel: string; defaultSpecModel: string;
  maxCallsBudget: int > 0; maxTokensBudget: int > 0;
```

```
  a3TimeoutMs: int > 0; z3TimeoutMs: int > 0;
  enableFlaky: boolean; enableAudit: boolean; enableBSynth: boolean }
```

200 `SettingsResponse` · **400** sai kiểu **hoặc có trường lạ**. Schema là `.strict()`: mọi trường như `apiKey` đều bị từ chối. Đây là **bất biến bảo mật**, đã có test bảo vệ.

> TODO backend: lưu bền vững (mock giữ trong bộ nhớ). Nên kiểm tra `defaultCoderModel`/`defaultSpecModel` có nằm trong danh mục `models`.

Phụ lục A — Bản đồ component ↔ API

```
src/lib/api/
  client.ts          apiRequest, ApiError, readNdjson, withQuery
  endpoints.ts       api.{datasets,runs,stats,cases,tickets,admission,evidence,revocation,re
  types.ts           DTO dùng chung FE   Route Handlers
  use-api-query.ts   hook đọc dữ liệu (huỷ request cũ, reload, setData sau mutation)

src/features/<feature>/
  <Feature>Screen.tsx    màn hình (client), lấy dữ liệu qua useApiQuery + api.*
  components/*.tsx       component trình bày, nhận dữ liệu qua props
src/app/**/page.tsx      chỉ bọc <Suspense> và render Screen tương ứng
src/ui/layout            PageHeader, Panel, MetricTile, Notice, Badge, QueryBoundary, ...
```

Feature	Screen	API
library	`LibraryScreen`	1.3, 3.1, 3.2
admission	`AdmissionScreen`	6.1–6.3, 3.3 (SSE)
annotate	`AnnotateScreen`	5.1–5.3
evidence	`EvidenceScreen`	7.1
revocation	`RevocationScreen`	8.1, 8.2
repair-gate	`RepairGateScreen` + `RepairGatePanel`	4.1, 4.3
results	`ResultsScreen`	9.1
experiments	`ExperimentsScreen`	10.1–10.4
fault-injection	`FaultInjectionScreen`	11.1, 11.2
settings	`SettingsScreen`	12.1, 12.2
history	`HistoryScreen`, `RunDetailScreen`	2.1, 2.3, 2.4, 2.6, 2.8
stats	`StatsScreen`	2.10
chat / workbench	`ChatTestbed`, `WorkbenchScreen`	1.3, 2.2, 2.5, 2.7, 2.9, 4.1, 4.4, 5.1

Phụ lục B — Kiểm thử hợp đồng

- `tests/helpers/api-routes.ts` định tuyến `fetch` trong test UI tới **chính các Route Handler**, nên test UI kiểm tra đúng hợp đồng ở tài liệu này.
- `tests/api/frontend-endpoints.test.ts` kiểm tra các endpoint mới (hàng đợi tất định, vé spent rời hàng đợi, RQ 404, validate config thực nghiệm, profile tiêm lỗi, review admission).
- Khi backend thật thay mock, chạy lại `pnpm test` để phát hiện lệch hợp đồng.